APS360 - FINAL REPORT
DOG BREED CLASSIFIER

Lijin Yu
Student 1004720901
lijin.yu@mail.utoronto.ca

Chen Hsin Chiang
Student# 1003888484
chenhsin.chiang@mail.utoronto.ca

Mengnan Tong
Student# 1004712545
mengnan.tong@mail.utoronto.ca

Emmett Bravakis
Student# 1005913273
emmett.bravakis@mail.utoronto.ca

ABSTRACT

Deep Learning has been a hot topic throughout this decade. With the discovery of neural networks, people are able to train the machine to do specific jobs beyond the human eye limitation, such as self-driving cars etc. For this project, the team implemented a Dog Breed Classifier using a Convolution Neural Network (CNN). An ideal model could potentially be used as a dog encyclopedia for people to identify and study specific dog breeds. This project has an established baseline model which serves as the minimal comparative criterion. The final model uses the 40-most-common-dog-breeds in 2022. The primary model has eight layers, comprising six convolutional and two linear layers. The model was trained and tested on previously compiled data as well as newly compiled data and the results were then analyzed to put into context, using the difficulty of the project to determine how successful the team believes the model is.

—-Total Pages: 9

1 INTRODUCTION

According to American Kennel Club, a not-for-profit organization that is dedicated to promoting responsible dog ownership and canine well-being, there are currently 200 officially registered dog breeds in the world American Kennel Club (2022). While there are many experts and dog owners who are savvy in dog breed classification (DBC), it can be difficult for common individuals to identify a dog's breed based on simple inspections or images. Therefore, an algorithm that can accurately identify a dog's breed based on images guides the user to gain more information based on limited source information.

The problem at hand is similar to some of the classification problems that the team has seen during lectures. Fundamentally, the goal is to classify an input image of a dog as one of the registered dog breeds. Deep learning is a well-suited approach for this problem due to the abundance of readily available and labeled datasets for training and validation. Similarly, acquiring new testing data is also straightforward and simple as the team can easily obtain photography of dogs anywhere. When a user runs the algorithm, the input images will be fed into the already-trained deep learning model and be classified as one of the registered dog breeds (Figure 1). After testing on different model architectures and transfer learning, the team aims to achieve a testing accuracy of 70% for 40 different dog breeds. Justifications for the chosen numbers are provided in Section 9.1.

On one hand, the algorithm is to assist a user in selecting a dog breed for purchase or adoption. Once selecting a picture of a dog that is appealing, the user can easily trace to the dog breed and acquire more information, thereby making a more informed decision in the end. On the other hand, the trained weights of the final model can be used in similar training problems via transfer learning, which helps facilitate other individuals to achieve a similar goal and save time.

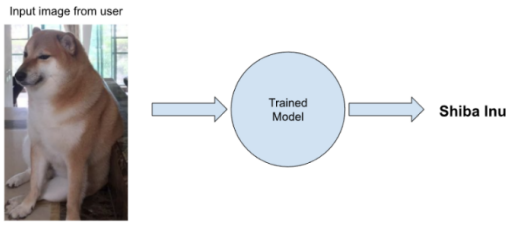

Figure 1: Steps to classify an input image as one of the registered dog breeds

2 BACKGROUND AND RELATED WORK

There are many pre-existing data sets that have pre-classified images that many people have used to test their models: Stanford Dogs is very applicable to our project. There are also other existing data sets such as the CalTech-UCSD Birds or Oxford Flowers for different categories. One such work for bird classification was studied by Lipnitskaya (2021) and they managed to achieve an accuracy of over 80% using transfer learning with multi-task learning from ResNet-50. For dog breed classification, there are plenty of source codes and tutorials on how to use it, such as TechVidan TV (n.d.). This is a step-by-step tutorial on how to create your own model, with source code and other resources. Other notable sample projects tried to use the same training data set, including 'Dog Superbreed Classification" by Chen et al.. They used many different models, and the best results came around 80% from using the pre-trained model MobileNetV2. The team's goal will be to produce our own CNN model that achieves similar success to many models that can be found online.

Some tests that are different yet have a similar goal are the dog DNA testing products. These DNA tests perform a variety of tasks related to your dogs' genetic makeup but they are also able to determine dog breeds. One obvious advantage of tests such as Embark's "Dog DNA Test" Embarkvet (n.d.) is that they are not limited by the visual appearance of dogs, which has been stated to be extremely flawed for mixed-breed dogs because "Over 90% of the dogs identified as having one or two specific breeds in their ancestry did not have their visually identified breeds as the predominant

breed in their DNA analysis" NCRC (n.d.). Another product is by EasyDNA, they also offer dog DNA tests to determine breed EasyDNDA (2023). This is not something our model will be able to be fairly compared to, but it is rather a more expensive alternative.

3 DATA PROCESSING

The team utilized the Stanford Dogs public data set (SDS), which contained 120 classes with 20,580 varying-sized RGB images. Since the dataset contained a variety of image sizes, the team had to process the image to ensure the CNN worked as intended. Furthermore, to improve the generalization and compatibility of the model, the team augmented the images. See Figure 2 for example images after processing. A brief summary of the overall procedure is given:

1. Resize the image to 256x256.
2. Randomly crop the image to a minimum of 224x224,
3. Random horizontal flip with a possibility of 0.3
4. Randomly rotate the image by 30 degrees
5. Randomly erase a small section of the image with scale=(0.01, 0.15), ratio=(0.1, 1.0), and probability = 0.2.

Figure 2: Examples of Processed Images

The team then decided to limit the number of classes to 40. See section 9.1 for detailed justifications. The team chooses the 40 classes based on the list of the most popular dog breeds in 2022 AKC (n.d.) so that it is more probable that the user inputs an image that is included in the trained classes and acquires an accurate result. Finally, the chosen classes range from 151 to 252 images. There are a total of 6858 images, with 5488 for the training set, 685 for the validation set, and 685 reserved for the testing set.

4 ARCHITECTURE

The primary model had eight layers in total with six convolutional layers and two fully connected layers. Each convolutional layer followed the following procedure:

output=pool(ReLU(batchNorm(conv(input))))

Where pool() is a 2x2 max pooling layer for extracting the sharpest features, ReLU() is the activation function for fast computation and reducing the vanishing gradient problem, batchNorm() is a regularization technique used for speeding up the training process, and conv() is the convolution operation. All the convolutional layers used a kernel size of three, as research had suggested that this filter size is cost-efficient for training Ihare (2020). At the last convolutional layer, dropout was applied to minimize model overfitting by nullifying the contribution of some neurons.

Then, the model was flattened into two fully connected layers:

output=ReLU(FC(input))

For similar reasons as above, a dropout layer was applied at the first fully connected layer. The last fully connected layer had an output size of 40 because the team was only classifying 40 different dog breeds. Detailed parameters and information about each layer are shown in Table 1 and Table 2:

Table 1: Summery for Convolutional Layers.

Layer Number	Input Size	Output Size	Input channel number	Output channel number	Kernal Size	Padding Size	Stride
1	224	111	3	30	3	0	1
2	111	54	30	30	3	0	1
3	54	26	30	60	3	0	1
4	26	12	60	60	3	0	1
5	12	5	60	120	3	0	1
6	5	1	120	120	3	0	1

Table 2: Summery for Fully Connected Layers.

Layer Number	Input Size	Output Size
1	120	80
2	80	40

The best model was trained with Cross Entropy loss and the Adam optimizer. Having conducted extensive testing, it was observed that Adam performed better than the SGD optimizer.

5 Baseline Model

The baseline model was a simple two-layer CNN model similar to the one constructed during labs. The architecture is shown in Figure 3.

```
class CNN_No_Norm(nn.Module): #model without batch normalization
    def __init__(self):
        super(CNN_No_Norm, self).__init__()
        self.name = "cnn_no_norm"
        self.conv1 = nn.Conv2d(3, 5, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(5, 10, 5)
        self.fc1 = nn.Linear(10 * 53 * 53, 256)
        self.fc2 = nn.Linear(256, 40)
    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 10 * 53 * 53)
        x = F.relu(self.fc1(x))
        #x = self.norm2(x)
        x = self.fc2(x)

        return x
```

Figure 3: Baseline Model Architecture

The model was trained with Cross Entropy loss and the SGD optimizer. The batch size was 128, the learning rate was 0.01, and the momentum was 0.9. The best-achieved accuracy was roughly 12.4%, seen in Figure 4. Given previous experiences using the same model, minimal tuning was required to prevent significant overfitting. Based on the training curves, it was observed that the accuracies had not reached a plateau. While training for more epochs may further increase accuracies, it was also important to optimize the training time.

6 Result

6.1 Training and Testing Results

To demonstrate the effectiveness and performance of the proposed model, the team presented the result with both quantitative and qualitative methods. For the quantitative measurements, the team calculated the accuracies of the model and plotted training accuracy against validation accuracy. The

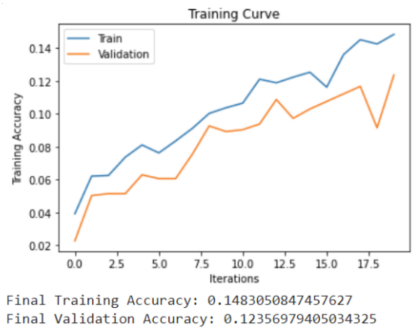

Figure 4: Training Curves for the Baseline Model

team used a batch size of 100, learning rate of 0.001, and weight decay of 0.003 for the final model. For detailed explanation, refer to section 7.

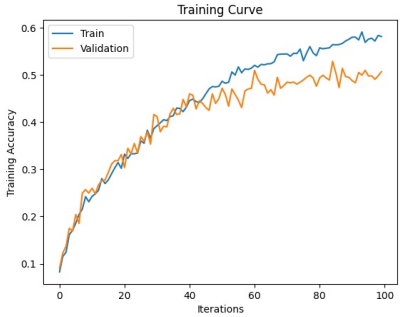

Figure 5: 100 Epoch Snapshot

The model had good fitting curves, however, the team encountered problems in using the free version of Colab: The standard GPU would automatically delete the runtime at its limit. With this in mind, the team trained the model for 30 epochs at a time and Figure 6 presents the training process for a larger training session of 270 epochs, where the top left corner is the beginning of the session and the bottom right corner ends with 270 epochs. From the graph, the curves matched the fitting in Figure 6 with small variations. From the middle of Figure 6, the model seems to reach its limit around 150 epochs, where both the accuracy stopped improving and the curves reached plateaus.

Given that the accuracy fluctuated, the team used the best result from the entire training session: the best training, validation, and test accuracy was 66.75%, 56.10%, and 52.91% with an average testing accuracy of 49.42%.

Next, the team printed the true and predicted labels to check the model qualitatively. The team kept the label in numerical form to keep the printed result clean and short. As shown in Figure 7, the predicted labels generally match the true label approximately half of the time. This confirms the quantitative result.

6.2 Results when evaluating on new data

Getting new data to test the model on was not difficult for our project. There is a plenitude of images of different dogs readily available on the internet a single Google search away. Twenty images of each dog breed the model was trained to detect were collected and compiled with the testing set already separated from the training and validation set our model was trained with. These images were selected by googling the dog breed name, followed by 'dog breed' (example: 'Maltese dog breed') then downloading the first twenty images that appeared on Google images, aside from the images that contained multiple different dog breeds which there were a few occurrences of. See Figure 8 for a sample batch of images from this compiled dataset. This seemed like a fair way to

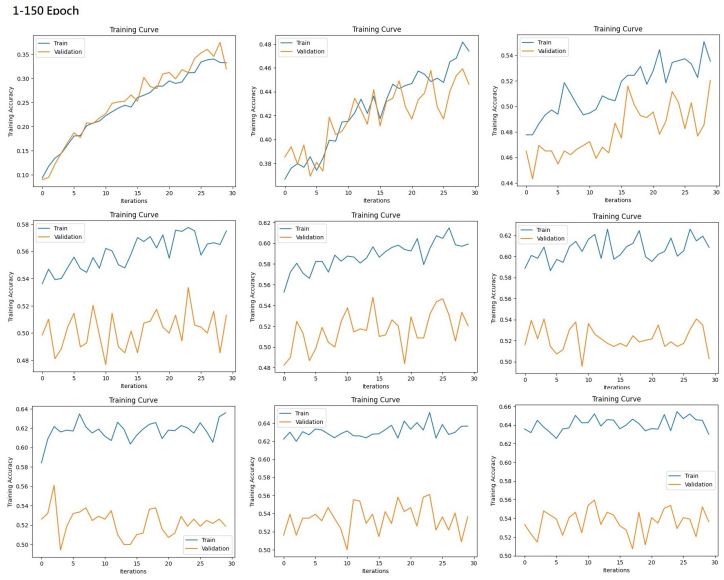

Figure 6: Snapshot of the training session with 270 epochs

```
Correct Labels
tensor([25,  6, 32,  8, 24, 27,  6,  4, 31,  4, 35, 24, 14, 15, 38, 17,  7, 16,
        19, 38, 28,  0, 13,  2, 33, 21, 37, 19, 20,  1, 11, 22,  7, 27, 27, 33,
        34, 38, 31,  3, 19, 32,  3, 36, 21,  5,  1,  7, 37,  1, 37, 11,  9, 15,
        23, 11, 31, 11, 24,  3,  4,  6, 19,  7, 21, 27,  0, 32, 10,  2, 37, 33,
        35,  5, 33, 20, 36,  6, 29, 30, 13, 14, 28,  9, 36, 24, 26, 25],
       device='cuda:0')
Predicted Labels
[tensor(25), tensor(30), tensor(30), tensor(8), tensor(24), tensor(27), tensor(6), tensor(25),
```

Figure 7: Snapshot of the true and predicted labels

collect the data as these images were not trained on and they were randomly selected. It was also possible to test the model against unlabelled data, as performed in our project presentation. The model can output its predicted labels for given unlabelled images.

Figure 8: Batch of new data

When testing the model on the new dataset, the data was processed the same way the testing data set from training was compiled. The model was then tested on this data to get the accuracy. The model performed below our goal of 70%, at about 50% accuracy. This was below our goal but lined up with the results from our training shown in section 6.1, which meant the model performed as expected on random data.

7 DISCUSSION

These results were much higher than the original baseline model (12%) but slightly lower than our goal. The training curve seemed to reach the ceiling around 67% after 150 epochs run and no improvement even after 270 epochs run. The model was underfitting, with training and validation curves close to each other. This could be a result of the model being not complex enough, especially given that the chosen dataset had lots of variations such as noisy background, different scales, angles of filming, and multiple dogs in one image. All of these made the model training process more difficult. The team had also tried to develop a more complicated model by expanding the filter size, the number of channels and the number of layers, but all these attempts faced RAM and GPU usage issues due to the limited computational power on Colab. Even after subscribing to premium GPU and high RAM in Colab, the team exceeded the maximum RAM usage for seven-layer CNN models.

The group had performed extensive research and experiments on other optimization methods such as hyperparameter tuning, etc. For example, different combinations of batch sizes plus learning rates were tested. Due to the large number of images in the training set, the team initially experimented with larger batch sizes but encountered RAM usage issues for trials with batch sizes larger than 400. The team observed that by increasing the batch size while gradually increasing learning rates, the accuracy increase was much slower: After training for 30 epochs, the trial with batch size = 250, learning rate = 0.003 achieved higher accuracy compared to the trial with batch size = 400, learning rate = 0.01 (Figure 9); However, too small of a batch size reduced the accuracy rate. From Figure 10, using a learning rate of 0.001, weight decay of 0.003, and 50 epochs, a batch size of 100 led to the best result. Finally, the team trained models with different batch sizes for more epochs to examine the degree of overfitting. It was observed that trials with batch sizes = 100 and 250 both had their validation accuracy plateauing around 50%, and the smaller batch size model was trained faster. Thus, a batch size of 100 was chosen for the final model.

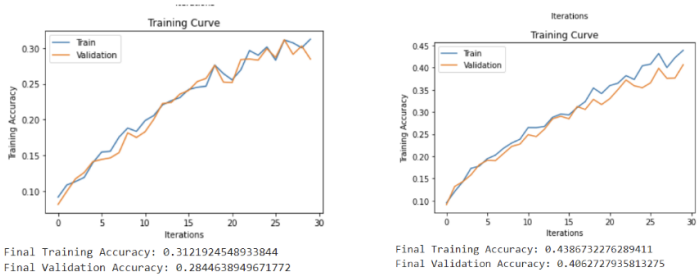

Figure 9: Left: batch size = 400. Right: batch size = 250

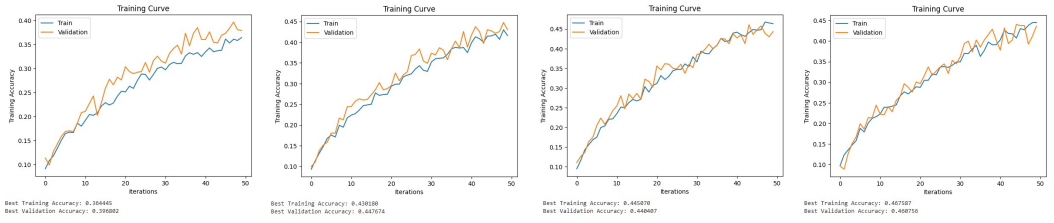

Figure 10: Training curve for learning rate = 0.001 and weight decay = 0.003, with batch size = 25/50/75/100, from left to right.

A larger weight decay helped to reduce overfitting while compromising the accuracy rate (Figure 11). By trial and error (Figure 12), the optimized value for Weight Decay was found to be 0.003. Furthermore, Dropout and Batch Normalization were added to the model to reduce overfitting and decrease running time. These two techniques did not adversely impact the validation accuracy.

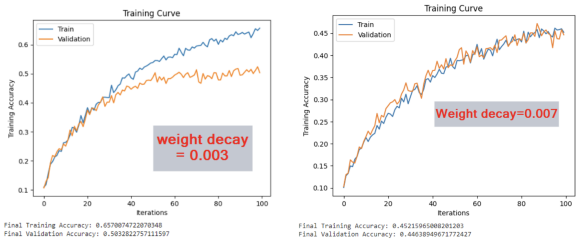

Figure 11: Training Curve for Weight Decay 0.003 and 0.007

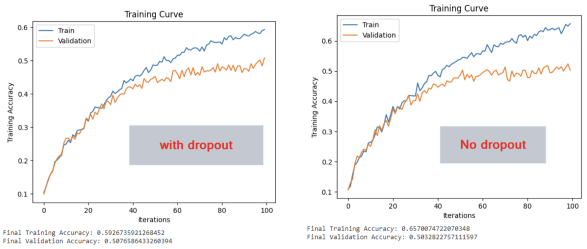

Figure 12: Training Curve for with and without Dropout

8 Ethical Consideration

The advantage of this project scope was that classifying dog breeds did not appear to have any obvious ethical considerations on paper, but there were still several factors that were considered. The team never falsify any results about the model. While DBC is not likely to result in any legal disputes, it is still important to note that the team never advertised unrealistic expectations that can be produced with our model. Additionally, when selecting the training and testing data, the team avoided including any images that contain personal information or people's likeness to prevent any negative consequences. Lastly, the team strived for transparency in our intentions. More specifically, the team included all our assumptions and design choices throughout the project.

9 Project Difficulty and Quality

9.1 Project Difficulty

The project was more difficult than anticipated. The initial transfer learning explorations indicated a high requirement for computational power at an early stage of the project. After training for 15 epochs based on AlexNet weights, the team obtained a validation accuracy of only 15%, and this trial took 70 minutes. Upon examination of other attempts at the DBC task, only residual networks (ResNets) such as Inception V3 achieved more than 70% validation accuracy for more than 100 classes Dabra (2020) Chen et al. Jamil (2020). With low-layer CNNs, most models could only achieve about 20% of testing/validation accuracy Shiangyi (2020) Chen et al.. Even with transfer learning using VGG16, which had about twice as many layers as the team's main model, the validation accuracy was under 40% Shiangyi (2020).

While using transfer learning models generally offered a higher output accuracy, this approach did not require much effort in addition to fine-tuning some parameters, further training the model using the selected dataset, and potentially applying some data pre-processing. Therefore, the team still chose the route of building a deep learning model from scratch. However, considering the difficulties discussed above, the goals and objectives were modified to accommodate the team's limited computational resources: Based on the testing results of the baseline model, the team chose to include the top 40 most popular dog breeds within the SDS and adjusted the target testing accuracy down from 80% to 70%.

With the new target accuracy and the number of classes, the required model for this project was still difficult to construct and train. As a comparison standard, all course laboratories could achieve more than 70% testing accuracy using 2-layer CNN models. Upon further testing of the project baseline model, the team was able to achieve a high training accuracy. However, the best validation accuracy of 2-layer CNN models still hovered around 12% (Figure 13).

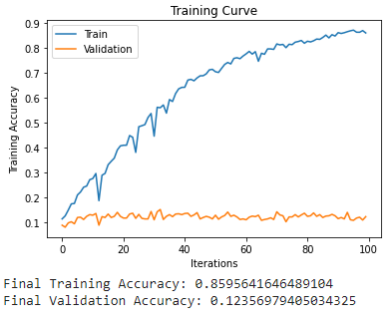

Figure 13: Best 2-layer CNN model training curve

9.2 Project Quality

While the model failed to meet the target of 70% testing accuracy, the model's performance was justified and evaluated as acceptable mainly based on computational power limits.

As discussed in section 7, the team's main model was limited to a 6-layer CNN. Since the main model's training accuracy plateaued around 60%, the team speculated that having a deeper model may promote a higher training accuracy, which also impacted the final validation accuracy.

Hyperparameter tuning was also limited by computational resources. For instance, despite observing that smaller batch sizes achieved similar accuracies (see section 7), testing of larger batch sizes (eg. 1000) was still desirable, given the total number of training images was large.

Finally, the team had already utilized data augmentation and regularization techniques where applicable. Based on testing results (Sections 6 and 7), the team concluded that while fine-tuning hyperparameters was indispensable to prevent overfitting and increase validation accuracy, constructing deeper and more sophisticated architectures was the most plausible mean to obtain higher accuracies. Overall, the team had attempted all methods within its capability to enhance the model performance.

10 Conclusion

In conclusion, the team identified the need for a dog breed classifier and proposed a solution path to achieve this goal. With background research and careful tuning of the CNN model's architecture, the team was able to achieve a testing accuracy of 52.91%. Although this did not achieve the initial objective, the team believed that this result was more than sufficient given our resources and setup. Furthermore, a detailed discussion is provided to explain and justify any ambiguity and controversy.

11 Colab Link

`https://colab.research.google.com/drive/1Z5xsuekRpxBtNy4qUEkY-Y2LJ71r-5IB?usp=sharing`

REFERENCES

American Kennel Club AKC. Breeds by year recognized. https://www.akc.org/press-center/articles-resources/facts-and-stats/breeds-year-recognized/, n.d. [Accessed: 15-Feb-2023].

American Kennel Club. Breeds by year recognized, 2022. URL https://www.akc.org/press-center/articles-resources/facts-and-stats/breeds-year-recognized/. [Accessed March 17, 2023].

Daniel Chen, Estefania Lahera, Hailey James, and Winnie Wang. Cs109 final project: Dog superbreed classification. URL https://hljames.github.io/dog-breed-classification/. [Accessed April 1, 2023].

Ayush Dabra. Dog breed classification using inceptionv3 cnn model on stanford dogs dataset, 2020. URL https://github.com/ayushdabra/stanford-dogs-dataset-classification. [Accessed April 8, 2023].

EasyDNDA. Dog dna tests, 2023. URL https://www.easydna.ca/dog-dna-tests/. [Accessed April 12, 2023].

Embarkvet. Embark dog dna test kits. https://embarkvet.com/, n.d. [Accessed: 15-Feb-2023].

Anuja Ihare. Significance of kernel size, 2020. URL https://medium.com/analytics-vidhya/significance-of-kernel-size-200d769aecb1. [Accessed March 17, 2023].

Danyal Jamil. Deep learning for dog breed classification, 2020. URL https://pub.towardsai.net/deep-learning-for-dog-breed-classification-77ef182a2509. [Accessed April 2, 2023].

Sofya Lipnitskaya. Data-driven approach for bird image classification, 2021.

National Canine Research Council NCRC. Visual breed identification. https://nationalcanineresearchcouncil.com/visual-breed-identification/, n.d. [Accessed: 15-Feb-2023].

Shiangyi. Dog breed image identification with cnn, 2020. URL https://saas.berkeley.edu/rp/dog-breed-id. [Accessed April 10, 2023].

Tech Vidvan TV. Dog's breed identification using deep learning. https://techvidvan.com/tutorials/dog-breed-classification/, n.d. [Accessed: 15-Feb-2023].